KAN it learn? Interpretable N-Body Physics

Hanson Sun
jsun33@students.cs.ubc.ca

Eitan Klinger
ekling01@students.cs.ubc.ca

Arthur Guo
aguo07@students.cs.ubc.ca

Abstract

Our project literally sucks and doesnt work

1 Introduction

Many scientific systems can be modeled as interacting objects governed by underlying physical laws. A key goal in machine learning for science is not only to predict these systems accurately, but also to recover interpretable structure from the learned model.

We study the n-body problem as a testbed for this goal. Given the initial states of several bodies, the task is to predict their future trajectories under pairwise forces. Prior work has shown that graph neural networks (GNNs) are a strong fit for this setting because they naturally represent objects as nodes and interactions as edges. In particular, Cranmer et al. 2020 showed that a GNN trained on simulated n-body dynamics can be combined with symbolic regression to recover analytical force laws.

A limitation of this approach is that the GNN itself is not directly interpretable. The learned representation is still a black box, and symbolic regression is a separate post-hoc step. Recent work on Kolmogorov-Arnold Networks (KANs) suggests a possible alternative. KANs replace standard MLP-style node activations with learnable univariate functions on edges, which can make the learned model easier to inspect. Liu, Wang, et al. 2024 showed that KANs can be effective while remaining more transparent than standard MLPs.

In this project, we ask whether this idea can be extended to relational physical systems. We introduce a hybrid Graph KAN (GKAN) architecture, where KAN layers replace MLP components inside a message-passing GNN. We evaluate this model on simulated n-body dynamics and compare it against standard GNN baselines, with a focus on both prediction quality and interpretability.

2 Related Work

This project builds on three main lines of work: graph-based physics learning, symbolic discovery, and interpretable neural architectures.

Cranmer et al. 2020 is the closest prior work. They trained GNNs on simulated n-body systems and used symbolic regression to recover physical laws from learned interaction functions. Their results show that graph models can discover useful structure, but interpretability comes only after a separate extraction step.

Other work has strengthened graph models for physical simulation. Thangamuthu et al. 2023 compared several physics-informed GNNs across multiple dynamical systems, while Dynami-CAL GraphNet (Sharma and Fink 2025) adds conservation and symmetry constraints directly into message

Course project for UBC CPSC 440/550, April 2026.

passing. These methods improve performance, but do not directly address whether the learned interactions are easy to interpret.

KANs provide a different route to interpretability. Liu, Wang, et al. 2024 showed that KANs can represent scientific relationships through learnable univariate functions, and KAN 2.0 adds multiplicative structure that is especially relevant for physical equations. Related continuous-time work such as KAN-ODEs (Koenig, Kim, and Deng 2024) and Graph KAN-ODEs (Cappi et al. 2025) further motivate combining KANs with dynamical systems and graph structure.

3 Methodology

We evaluate GKANs on synthetic n-body simulation data generated with a leapfrog integrator. The main dataset uses Newtonian gravity,

$$a_i = G \sum_{j \neq i} m_j \frac{r_j - r_i}{\|r_j - r_i\|^3} \tag{1}$$

with additional force laws for generalization tests. Each example consists of initial positions, velocities, and masses, with targets given by the future trajectories.

[HS: we need to provide the formulas of the force laws we used somewhere, maybe in appendix]
[HS: we also should provide baseline specification somewhere, ie how many layers, width, etc]

We compare two models: a graph convolutional network baseline following Cranmer et al. 2020, and a GKAN that replaces selected MLP components with KAN layers. Both models are trained on the same data, with the same loss and optimization setup, to keep the comparison fair.

We evaluate models using trajectory prediction error on held-out simulations. We also examine interpretability through sparsity, visual structure of learned edge functions, and how easily the model can reflect physically meaningful relationships.

We organize our experiments around a few questions: whether GKANs can match GNN performance at similar parameter counts, whether sparse regularization makes learned functions easier to interpret, whether smaller KAN-style architectures are sufficient, whether physically motivated feature engineering helps, and whether the same trends hold under different force laws such as Hooke's law.

[HS: will probably skip multkans, dont think i got anything useful there. can include if theres space]

[HS: state what we expect to learn] We expect the node KAN to learn how to aggregate messages into a final acceleration vector, while the edge KANs learn the pairwise interaction functions. We hypothesize that regularization will encourage sparsity and interpretability, and that smaller KAN architectures may still perform well if they capture the right inductive biases.

3.1 Baseline GNN

3.2 Regularization

Sparsification can be achieved in KAN layers through L1 regularization of weights, as in traditional MLP layers, though an additional entropy regularizer is often needed to make sufficiently sparse KANs Liu, Wang, et al. 2024. This suite of experiments serves to both validate regularization patterns found in traditional KANs, as well as find ideal regularization hyperparameters to encourage sparse and interpretable networks. To do so, we run a sweep through different magnitudes of L1 regularization, entropy regularization, and both combined, comparing the models to each other, as well as a control with no regularization. Doing so helps identify the efficacy of different regularizations in creating sparser models.

Each GKAN network uses a node network of dimensions [7, 8, 2], a message network of dimensions [10, 12, 12, 2], and is trained over 50 epochs. Once all models are trained, they are compared in both training loss, validation loss against a test set, as well as visually compared for how sparse the networks are. The ideal regularization approach is determined by which hyperparameters provide a relatively sparse and interpretable network without sacrificing performance on the validation set.

Idea: try to test out and find the best regularization hyperparameters to encourage sparse networks. all experiments done using gravity, n_train = 500, n_val = 50, msg_width = msg_width: [10, 12, 12, 2], node_width: [7, 8, 2], over 50 epochs using adam

- control with no regularization
- sweep of l1 regularization: 1e-3, 1e-4, 1e-5 **[EK: maybe drop the 1e-5, since it's the only case using that number and isn't important]**
- sweep of entropy regularization: 1e-3, 1e-4
- sparse initialization
- sweep of both l1 and entropy regularization: 1e-4, 1e-3, 1e-1, 1e1, 1e2

[HS: justify choice of regularization (L1 and entropy)]

3.3 How small KAN we get?

The Kolmogorov-Arnold (KA) representation theorem states that any multivariate function can be decomposed into a finite number of univariate functions, combined via addition and function composition. **[AG: Who do I cite here? K and A from the 50s? Also, re-stating this stuff isn't actually that useful...]**

$$f(x_{1\ldots n}) = \sum_{i=0}^{2n} \Phi_i \left(\sum_{j=1}^{n} \phi_{i,j}(x_j) \right)$$

This suggests the existence of models with significantly fewer parameters that can still represent the function we want to learn. In this section, we design and evaluate several smaller models to determine if the GKAN can perform well without extreme overparameterization.

To design a smaller model, we work backwards from a proposed "feasible route" of function compositions and additions that yields the desired force law. In other words, we set layer widths and depths based on the number of arithmetic steps needed to decompose the desired force law into univariate functions by hand. We then observe whether the model is capable of recovering the desired force law by "undoing" our decomposition.

To illustrate this process, Figure 1 is one example of a feasible route. In this example, the message network consists of 4 layers ($10 \to 6 \to 3 \to 4 \to 2$) because the acceleration vector can be computed in 4 arithmetic steps; the node update network consists of just 1 layer ($7 \to 2$) because it only needs to aggregate acceleration vectors from the messages. **[AG: I suppose we should explain the other 3 experiments in an appendix.]**

In this section, we also experiment with feature engineering by augmenting the dataset with computed features. Specifically, we are interested in computed features that allow us to "skip steps" during the decomposition process. For example, in the decomposition described by Figure 1, it is necessary to compute the intermediate values Δx and Δy in the first layer. By providing these as inputs, we can eliminate 1 layer.

In total, we designed and evaluated 4 smaller models with several combinations of computed features as listed in Figure 2. For comparison, the baseline GNN model based on Cranmer et al. 2020 has 300 hidden MLP layers and 427 002 parameters.

The node update network always consists of 1 layer ($7 \to 2$). The models are all trained for 50 epochs on the same dataset, which consists of 200 training trajectories and 20 validation trajectories. For exact details on the data generation process and training regime, as well as an outline of the feasible route for each model, refer to the appendices. **[AG: Use a ref here.]**

4 Results

[HS: maybe have one big table summarizing losses for different configs (include maximal model somehow?)]

input	m_i, $s_{x,i}$, $s_{y,i}$, $v_{x,i}$, $v_{y,i}$, m_j, $s_{x,j}$, $s_{y,j}$, $v_{x,j}$, $v_{y,j}$
1	$h_{1,1} = s_{x,j} - s_{x,i} \propto \Delta x$ $h_{1,2} = s_{y,j} - s_{y,i} \propto \Delta y$ $h_{1,3} = m_j + s_{x,j} - s_{x,i} \propto m_j + \Delta x$ $h_{1,4} = m_j - s_{x,j} + s_{x,i} \propto m_j - \Delta x$ $h_{1,5} = m_j + s_{y,j} - s_{y,i} \propto m_j + \Delta y$ $h_{1,6} = m_j - s_{y,j} + s_{y,i} \propto m_j - \Delta y$
2	$h_{2,1} = h_{1,1}^2 + h_{1,2}^2 \propto \lVert r \rVert^2$ $h_{2,2} = h_{1,3}^2 - h_{1,4}^2 \propto m_j \Delta x$ $h_{2,3} = h_{1,5}^2 - h_{1,6}^2 \propto m_j \Delta y$
3	$h_{3,1} = h_{2,1}^{-1.5} + h_{2,2} \propto \lVert r \rVert^{-3} + m_j \Delta x$ $h_{3,2} = h_{2,1}^{-1.5} - h_{2,2} \propto \lVert r \rVert^{-3} - m_j \Delta x$ $h_{3,3} = h_{2,1}^{-1.5} + h_{2,3} \propto \lVert r \rVert^{-3} + m_j \Delta y$ $h_{3,4} = h_{2,1}^{-1.5} - h_{2,3} \propto \lVert r \rVert^{-3} - m_j \Delta y$
4	$h_{4,1} = h_{3,1}^2 - h_{3,2}^2 \propto \lVert r \rVert^{-3} m_j \Delta x$ $h_{4,2} = h_{3,3}^2 - h_{3,4}^2 \propto \lVert r \rVert^{-3} m_j \Delta y$
output	$a_{x,i} = h_{4,1}$, $a_{y,i} = h_{4,2}$

Figure 1: A feasible route. **[AG: Need a better way to present this.]**

experiment name	message network	parameters	computed features
`small`	4 layers ($10 \to 6 \to 3 \to 4 \to 2$)	2220	
`small_rel_pos`	3 layers ($12 \to 5 \to 4 \to 2$)	2020	Δx, Δy
`small_rel_dsq`	3 layers ($13 \to 5 \to 4 \to 2$)	2112	Δx, Δy, $\lVert r \rVert^{+2}$
`small_rel_idc`	3 layers ($13 \to 4 \to 4 \to 2$)	1824	Δx, Δy, $\lVert r \rVert^{-3}$

Figure 2: The smaller models.

4.1 Regularization

[EK: insert table with training loss, val loss at 50 epochs? TODO: load respective checkpoints to get actual numbers, and plot the losses on a graph. make a script so we can do this for other models]

I think most of this actually goes in discussion?

- all ts sucks ass!!!!
- regularization did not encourage sparsity
- just made model learn worse as regularization increased (regularization penalty overwhelming loss function?)
- like no discernible diff :(
- sparse init just sucked **[EK: barely worth mentioning]**
- regularization combined 1e1, 1e2 literally did not learn like the training error stayed flat. same for sparse init

TODO: generate loss plots overlaid, regenerate visualizations for experiments that are missing them

4.2 How small KAN we get?

Recall that we designed and evaluated 4 smaller models. The training loss and validation loss are summarized in Figure 3.

The `small` GKAN model without any computed features performs about 1 order of magnitude worse compared to the baseline GNN. This alone is quite impressive when considering the difference in model size.

Furthermore, by introducing simple computed features such as Δx and Δy (which would not be unreasonable to include, even without *a priori* knowledge of the desired force law), we observe

experiment name	training loss	validation loss
`baseline`	1.592×10^{-2}	1.281×10^{-2}
`small`	1.130×10^{-1}	1.185×10^{-1}
`small_rel_pos`	1.873×10^{-2}	1.639×10^{-2}
`small_rel_dsq`	1.960×10^{-2}	1.172×10^{-2}
`small_rel_idc`	2.594×10^{-2}	2.073×10^{-2}

Figure 3: The smaller models.

significantly improved performance on par with the baseline—with the additional benefit of a 99.5% reduction in model size. This suggests that the GKAN could be a highly efficient architecture for domain-informed use cases.

[AG: Include loss plots and rollouts here?]

5 Discussion (REAL)

5.1 Regularization

None of the different regularizations used resulted in networks sparse enough to be interpretable through visual inspection. Even using multipliers of 10 or 100 for the overall regularization only produced marginally sparser GKANs compared to the no-regularization control, while being unable to learn due to the regularization loss overpowering prediction loss. It was expected that the L1-norm and entropy regularizations alone would be insufficient, though even combined, as outlined in Liu, Wang, et al. 2024, results are disappointing. This may be explained by:

1. The gravitational n-body problem requiring dense networks to accurately represent the dynamic system. Though Liu, Ma, et al. 2024 provides an example of a symbolic KAN modeling gravitational force between two bodies in a small, interpretable network, indicating that the gravitational forces do not require dense networks. **[EK: I HATE THIS I HATE THIS THIS]**
2. Constant regularization does not allow for balance between learning and sparsity. The experiments did not increase regularization magnitudes over the learning period, which may have done a better job of letting the model optimize for accurate predictions first, then learn a sparse representation after.

Why does all ts suck so much ass? And why no sparse?

- maybe gravity just requires dense networks to model without using feature engineering; having to learn relative positions, multiplication and stuff might just be too hard for our little guys
- maybe the nature of KANs just doesn't lend itself to learning networks
- maybe if we had a bajillion more epochs it would actually fix itself
  [HS: igb...] [HS: maybe think about potential improvements?]

5.2 How small KAN we get?

[AG: Could do interpretability analysis. Pass synthetic inputs through the network to see the relationships empirically (sanity check). Then try symbolic regression per-layer and analytically composing them to see if it recovers the force law?]

[AG: The following is outdated.] Looking at the visualizations, it seems that the models without any augmentation did not learn as effectively. The message and node update networks are both quite dense, which suggests that they did not learn to ignore distractors such as velocity. By augmenting the dataset with `rel_pos`, we introduce an inductive bias that is apparently sufficient to "push" the model in the right direction, and interestingly, we can start to see sparsity in the networks. Refer to the visualizations for `minimal` and `minimal_rel_pos` to see this effect in action. **[HS: link to KAN 2.0 paper]**

I have no idea what mathematical relationships it actually learned. `/shrug` **[HS: show image]**

[HS: plots and rollouts? maybe keep in appendix]

5.3 How small KAN we get?

[AG: Could do interpretability analysis. Pass synthetic inputs through the network to see the relationships empirically (sanity check). Then try symbolic regression per-layer and analytically composing them to see if it recovers the force law?]

[AG: The following is outdated.] Looking at the visualizations, it seems that the models without any augmentation did not learn as effectively. The message and node update networks are both quite dense, which suggests that they did not learn to ignore distractors such as velocity. By augmenting the dataset with `rel_pos`, we introduce an inductive bias that is apparently sufficient to "push" the model in the right direction, and interestingly, we can start to see sparsity in the networks. Refer to the visualizations for `minimal` and `minimal_rel_pos` to see this effect in action. **[HS: link to KAN 2.0 paper]**

I have no idea what mathematical relationships it actually learned. `/shrug` **[HS: show image]**

[HS: plots and rollouts? maybe keep in appendix]

5.4 KAN't Optimize

[AG: discuss future improvements like pruning low-weighted edges every few epochs] [HS: should be in regularization section.]

Beyond interpretability, a major challenge in our experiments was optimization. Although GKANs are expressive in principle, they were consistently harder to train than comparable MLP-based GNN baselines. Loss often decreased slowly, converged to worse minima, or was highly sensitive to initialization and optimizer choice. We hypothesize this is mainly due to two factors: (1) optimization difficulty of spline-based edge functions, and (2) interaction with message passing leading to irregular gradients.

Optimization difficulty of spline-based edge functions. Each KAN edge learns a nonlinear spline, so the model must learn both the function value and its local gradient behavior. Flat spline regions can suppress gradients, while steep regions can create unstable updates. This becomes more difficult when many KAN layers are composed.

For gravity, the learned interaction also involves inverse-distance terms, which are asymptotic and poorly conditioned for spline approximation. This likely made optimization especially difficult. In support of this, enforcing a minimum separation distance between bodies noticeably improved training, whereas unconstrained runs reduced loss by less than 5%. Moreover, replacing gravity with the better-conditioned Hooke's law produced much larger loss decreases of roughly 90%.

Interaction with message passing. Message passing may further amplify these issues, since errors in learned edge functions propagate across the graph. Prior work has identified oversmoothing, oversquashing, and gradient degradation in deeper GNNs (Arroyo et al. 2025). Our results are consistent with this behavior. We tested Adam, L-BFGS, warmup schedules, and alternating optimization, but none consistently improved training. This suggests that optimization may be intrinsically challenging for GKANs in message-passing settings.

[AG: discuss potential for physically informed loss function (e.g., incorporating conservation laws or physical constraints into the loss function, hamiltonian loss functions, etc). This may partially explain poor learning results over all]

Physically-informed loss function. A common approach to improving learning in dynamical systems is to incorporate physical structure directly into the loss function. In particular, Hamiltonian and Lagrangian graph neural networks enforce conservation laws by parameterizing or constraining the system dynamics through a Hamiltonian function. More recent physics-informed GNNs extend this idea by explicitly enforcing conservation of linear and angular momentum through equivariant message passing (Sharma and Fink 2025). In our setting, such constraints could reduce the effective

hypothesis space of the model and encourage solutions consistent with known mechanics, potentially improving both optimization stability and long-term rollout accuracy in GKANs.

[HS: adaptive grid updates i guess? dont think this would matter that much tbh]

6 Conclusion

References

Álvaro Arroyo, Alessio Gravina, Benjamin Gutteridge, Federico Barbero, Claudio Gallicchio, Xiaowen Dong, Michael Bronstein, and Pierre Vandergheynst (2025). *On Vanishing Gradients, Over-Smoothing, and Over-Squashing in GNNs: Bridging Recurrent and Graph Learning*. arXiv: 2502.10818 [cs.LG]. URL: https://arxiv.org/abs/2502.10818.

Riccardo Cappi, Paolo Frazzetto, Nicolò Navarin, and Alessandro Sperduti (2025). "Unveiling the Actual Performance of Neural-based Models for Equation Discovery on Graph Dynamical Systems." arXiv: 2508.18173 [cs.LG]. URL: https://arxiv.org/abs/2508.18173.

Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho (2020). *Discovering Symbolic Models from Deep Learning with Inductive Biases*. arXiv: 2006.11287 [cs.LG]. URL: https://arxiv.org/abs/2006.11287.

Benjamin C. Koenig, Suyong Kim, and Sili Deng (Dec. 2024). "KAN-ODEs: Kolmogorov–Arnold network ordinary differential equations for learning dynamical systems and hidden physics." *Computer Methods in Applied Mechanics and Engineering* 432, p. 117397. ISSN: 0045-7825. DOI: 10.1016/j.cma.2024.117397. URL: http://dx.doi.org/10.1016/j.cma.2024.117397.

Ziming Liu, Pingchuan Ma, Yixuan Wang, Wojciech Matusik, and Max Tegmark (2024). *KAN 2.0: Kolmogorov-Arnold Networks Meet Science*. arXiv: 2408.10205 [cs.LG]. URL: https://arxiv.org/abs/2408.10205.

Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark (2024). *KAN: Kolmogorov-Arnold Networks*. arXiv: 2404.19756 [cs.LG]. URL: https://arxiv.org/abs/2404.19756.

Vinay Sharma and Olga Fink (2025). "Dynami-CAL GraphNet: A Physics-Informed Graph Neural Network Conserving Linear and Angular Momentum for Dynamical Systems." arXiv: 2501.07373 [cs.LG]. URL: https://arxiv.org/abs/2501.07373.

Abishek Thangamuthu, Gunjan Kumar, Suresh Bishnoi, Ravinder Bhattoo, N M Anoop Krishnan, and Sayan Ranu (2023). *Unravelling the Performance of Physics-informed Graph Neural Networks for Dynamical Systems*. arXiv: 2211.05520 [cs.LG]. URL: https://arxiv.org/abs/2211.05520.

[HS: we need to add github links, rollout, charts, images]

A Supplementary material

This stuff doesn't count towards your page limit.

B What goes in the report

Part of your grade is based on the contribution of your project: did you do something meaningful in your project, that adds something new to the world? Again, we don't expect you to necessarily make a high-impact totally novel result, and it's fine if you end up with a negative result, but you should have something here. You should make sure that the contribution of your project is clear in the introduction of your paper, and that the rest of the writeup clearly demonstrates that you actually made this contribution.

A large portion of your grade is based on writing a clear and structured paper that addresses the relevant questions that would be asked of a generic scientific/engineering publication. Your writeup should look more or less like a scientific paper as published at, say, NeurIPS, or a NeurIPS workshop. To achieve that, most papers should use something like the following traditional outline:

1. Introduction: Clearly state the problem being addressed. **Explain why it is an important problem to work on**. At a high level, briefly summarize what the limitations of existing approaches that your work will be addressing, and what the contribution of your project is.
2. Related Work: Identify at least three publications on related topics; usually these will be papers that have worked on slightly different problems or papers that have proposed an approach to your problem that is not fully satisfactory. For each paper, briefly say either how the problem addressed is related and/or different (if they address a different problem) or why it doesn't solve the problem you are working on (if it addresses the same problem). Sometimes it makes more sense to put this later in the paper, but usually it's here.
3. Description and justification of what you did, divided up (possibly in multiple sections and/or sub-sections). There's a lot of flexibility here, and it will depend on the type of project you are doing. For example, if you're applying standard machine learning methods to a new dataset or doing a Kaggle competition, you could have one (sub-)section describing the dataset and why you think machine learning could help, and one (sub-)section stating the methods you will try and why you think these are appropriate methods (you don't necessarily have to go into detail describing the methods). If you're extending an existing technique, you could have one sub-section describing the existing technique, and one sub-section for each of the extensions you explored. If your project has a theoretical component, you might have one sub-section discussing the assumptions, one sub-section describing the results, and one sub-section describing implications.
4. Experiments and/or analysis (if you have an experimental component): Describe each experiment that you did. Say what each experiment is trying to test. Ideally, each experiment should only try to test one thing and you should control for as many other factors as possible. Subsequently, summarize the result of your experiment, in both the text and in a nice visual form such as a figure; **a table with a huge list of numbers is often not the best way to summarize information**, but sometimes a well-designed table (especially with some colour-coding) is still your best bet.
5. Discussion and future work: State the main conclusions you obtained from this course project. List at least one strength and one weakness of your contribution. Briefly describe what you might do with more time.

An example outline for a perspective paper might be something like

1. Introduction: Clearly state the problem being addressed. Explain why it is an important problem. At a high level, briefly summarize the history of the works that will be discussed in the project.
2. Review: Go through the different works in some logical order, such as chronologically or by going from simple to complex models. Don't just list the methods, but say how they relate to each other (going through the strengths/weaknesses of the different methods, both in comparison to each other and compared to an ideal method that solves the problem).
3. Discussion: Discuss the trends that have occurred over time. Speculate about where the next steps in the trend could lead. Point out issues that are not properly addressed by existing methods. State some interesting directions to explore, or opportunities to use existing tools in new applications.

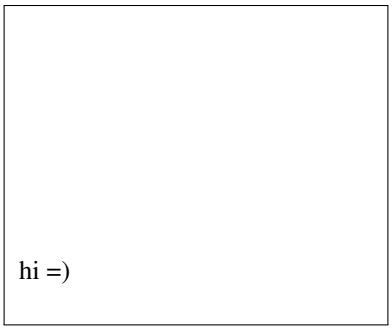

Figure 4: Sample figure caption.

Table 1: Sample table title

Part		
Name	Description	Size (μm)
Dendrite	Input terminal	$\sim$100
Axon	Output terminal	$\sim$10
Soma	Cell body	up to 10^6

You definitely don't have to stick to these exact structures; many of my papers don't. But if you're not using this format (…or if you are), you should make sure what you're doing makes sense, and answers the important questions about your project.

Writing style Mark has some advice for writing here, most of which I agree with: `https://www.cs.ubc.ca/~schmidtm/Courses/Notes/writing.pdf`. (He has a bunch of minor grammatical errors in that document, though. :/)

You're not going to lose points for making some minor grammatical mistakes or anything like that – the published literature is full of them! You will lose points if there are grammatical errors to the level of making your work hard to understand. More importantly, you will also lose points if your writing doesn't convey what you did in a clear, easy-to-understand way. Note that this involves a lot more than just having a good grasp of written English.

C How to format stuff

C.1 Citations

Something a lot of people don't know at first: use `\citet{ref}` (or `\textcite`) if the citation plays a grammatical role in the sentence, e.g. "**transformers** demonstrated that …." Use `\citep{ref}` (or `\parencite`) if it doesn't, e.g. "Machine learning is fun (**ml-is-fun**)."

C.2 Figures

See Figure 4 for how to include a figure.

C.3 Tables

Table captions go *above* the table, because that's the usual style, idk. There's an example in Table 1. Avoid having too many lines, and in particular don't just put them between every table cell; put them where they help readability.

C.4 Math

Using display math in bare TeX commands, like `$$f(x)$$`, will mess up the line numbers. (Inline math like `$f(x)$` is fine.) Instead use `\[ f(x) \]` or `\begin{equation*} f(x) \end{equation*}`, like this:

$$\sum_i a_i = 2.$$

(You shouldn't use `$$` anyway; see this discussion and this one for more.)

C.5 Supplementary Material

You can include extra information in appendices, like Appendix A. You don't have to if you don't want to.

Please *don't* include code in your writeups, unless you did something particularly cool and want to *briefly* describe the way something works as a contribution (this should be relatively unusual). Please instead link to it somewhere, preferably a GitHub repo or similar, especially if the code is a significant contribution of your project.

D Discussion

In the end, you should give us an A++++.

Acknowledgments

You can acknowledge useful discussion with other people who aren't coauthors here (or leave the section out). Typically you'd also put funding here, acknowledgements for computing clusters, etc.

References

Álvaro Arroyo, Alessio Gravina, Benjamin Gutteridge, Federico Barbero, Claudio Gallicchio, Xiaowen Dong, Michael Bronstein, and Pierre Vandergheynst (2025). *On Vanishing Gradients, Over-Smoothing, and Over-Squashing in GNNs: Bridging Recurrent and Graph Learning*. arXiv: 2502.10818 [cs.LG]. URL: https://arxiv.org/abs/2502.10818.

Riccardo Cappi, Paolo Frazzetto, Nicolò Navarin, and Alessandro Sperduti (2025). "Unveiling the Actual Performance of Neural-based Models for Equation Discovery on Graph Dynamical Systems." arXiv: 2508.18173 [cs.LG]. URL: https://arxiv.org/abs/2508.18173.

Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel, and Shirley Ho (2020). *Discovering Symbolic Models from Deep Learning with Inductive Biases*. arXiv: 2006.11287 [cs.LG]. URL: https://arxiv.org/abs/2006.11287.

Benjamin C. Koenig, Suyong Kim, and Sili Deng (Dec. 2024). "KAN-ODEs: Kolmogorov–Arnold network ordinary differential equations for learning dynamical systems and hidden physics." *Computer Methods in Applied Mechanics and Engineering* 432, p. 117397. ISSN: 0045-7825. DOI: 10.1016/j.cma.2024.117397. URL: http://dx.doi.org/10.1016/j.cma.2024.117397.

Ziming Liu, Pingchuan Ma, Yixuan Wang, Wojciech Matusik, and Max Tegmark (2024). *KAN 2.0: Kolmogorov-Arnold Networks Meet Science*. arXiv: 2408.10205 [cs.LG]. URL: https://arxiv.org/abs/2408.10205.

Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark (2024). *KAN: Kolmogorov-Arnold Networks*. arXiv: 2404.19756 [cs.LG]. URL: https://arxiv.org/abs/2404.19756.

Vinay Sharma and Olga Fink (2025). "Dynami-CAL GraphNet: A Physics-Informed Graph Neural Network Conserving Linear and Angular Momentum for Dynamical Systems." arXiv: 2501.07373 [cs.LG]. URL: https://arxiv.org/abs/2501.07373.

Abishek Thangamuthu, Gunjan Kumar, Suresh Bishnoi, Ravinder Bhattoo, N M Anoop Krishnan, and Sayan Ranu (2023). *Unravelling the Performance of Physics-informed Graph Neural Networks for Dynamical Systems*. arXiv: 2211.05520 [cs.LG]. URL: https://arxiv.org/abs/2211.05520.